

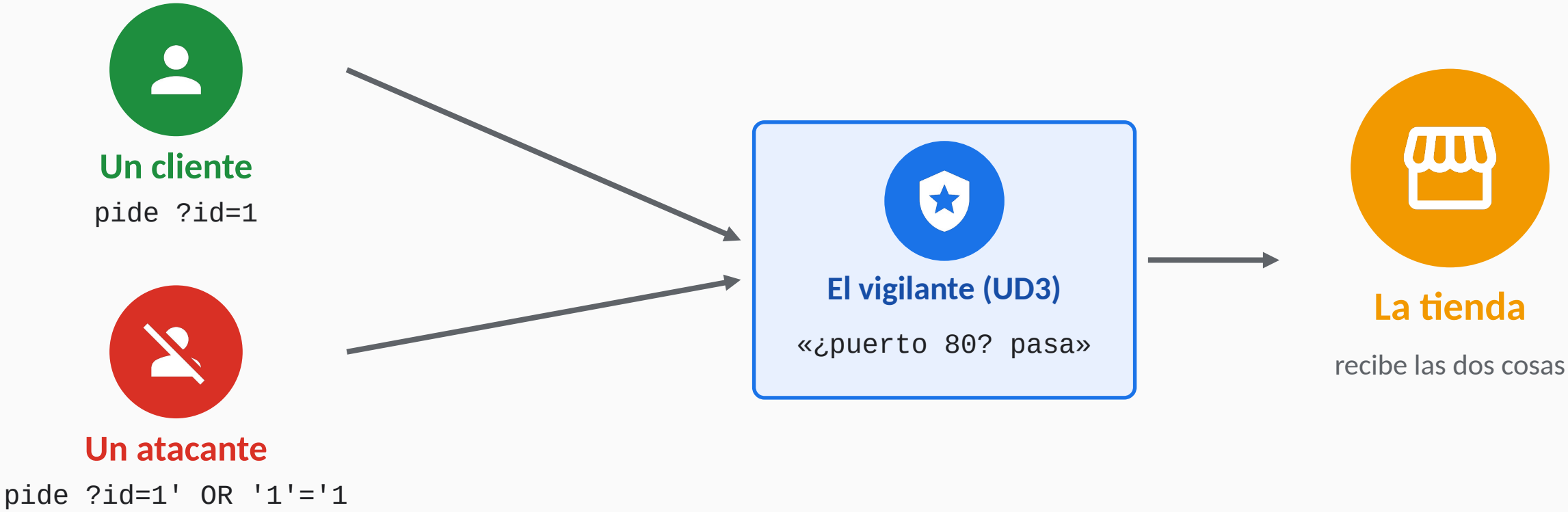
SAD · UNIDAD 5

IDS, IPS y WAF: vigilar lo que entra

Cámaras que avisan, un vigilante que corta el paso y un dependiente que lee cada pedido

EL PROBLEMA

Por la puerta de la tienda entra de todo



LO QUE FALTA — Mirar DENTRO de lo que trae cada visitante, no solo por qué puerta entra.

LA IDEA

Tres ayudantes nuevos para el edificio



IDS

Las cámaras

Graban, reconocen lo raro y AVISAN. No paran nada.

`Suricata (modo IDS)`



IPS

El vigilante que corta

Mira las cámaras y, si ve algo, CORTA EL PASO en ese momento.

`Suricata (modo IPS)`



WAF

El dependiente

Lee cada pedido de la tienda y RECHAZA los trucados.

`ModSecurity + OWASP CRS`

SIGLAS — IDS = sistema de detección de intrusos · IPS = de prevención · WAF = cortafuegos de aplicaciones web.

La diferencia es dónde se pone

IDS · mira una copia



IPS · en medio del camino



	IDS	IPS
¿Para el ataque?	No, avisa	Sí, tira el paquete
Si se cae...	no pasa nada	puede cortar el tráfico
Si se equivoca...	una alerta de más	deja fuera a un cliente
En el laboratorio	./lab.sh ids	./lab.sh ips

CÓMO RECONOCEN UN ATAQUE

Tres maneras de saber que algo es raro

Firmas

El cartel de «se busca»: si coincide, alarma.

`Ej.: en la URL aparece «union»`

✘ No ve un ataque nuevo

Anomalías

No sabe quién es, pero ve algo raro.

`Ej.: 100 puertas probadas en 1 segundo`

✘ Más falsas alarmas

Reputación

Una lista de IPs con antecedentes.

`Ej.: IPs conocidas de botnets`

✘ Hay que tenerla al día

HOY — Firmas: escribimos nosotros las reglas de Suricata.

EL LABORATORIO

Cinco máquinas en Docker: docker-sad-ud56



terminal · en la carpeta docker-sad-ud56

```
./lab.sh arrancar      # la primera vez tarda unos minutos
./lab.sh ayuda         # todas las órdenes
./lab.sh internet      # entrar en el atacante (sales con exit)
./lab.sh alertas       # lo que ha visto Suricata
./lab.sh bloqueos      # lo que ha visto el WAF
```

LA TIENDA — http://203.0.113.254 → el firewall la manda al WAF → el WAF a la tienda.

UNA REGLA DE SURICATA

Si ves ESTO, haz ESTO

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB ping desde Internet"; itype:8; sid:1000001; rev:1;)
```

ACCIÓN — qué hago

CABECERA — qué tráfico miro

OPCIONES — qué tiene que tener y cómo se llama

Leída en voz alta: «**avisa** de los **ping que vengan de Internet a nuestra red**; la alerta se llama **LAB ping desde Internet** y es la número 1000001».

Va en firewall/suricata/local.rules · una regla por línea · lo que empieza por # es un comentario

ES EL 👀 EJEMPLO — Ya está en local.rules del laboratorio. Las ✍️ las escribes tú debajo.

LA CABECERA

Pieza a pieza

Pieza	En el ejemplo	Qué significa	Otras que usarás
Acción	`alert`	avisa (y apunta en eve.json)	drop = tira el paquete (solo IPS)
Protocolo	`icmp`	qué tipo de tráfico	tcp · udp · http (entiende la web)
Origen	`$EXTERNAL_NET any`	de dónde viene: IP y puerto	203.0.113.50 any
Sentido	`->`	de origen a destino	<> = en los dos sentidos
Destino	`$HOME_NET any`	a dónde va: IP y puerto	$HOME_NET 22 · any any

$HOME_NET

nuestras redes (en el laboratorio, la DMZ).
$EXTERNAL_NET = todo lo demás.

Ojo con la tienda

Suricata mira ANTES del DNAT: el destino es 203.0.113.254. Para la web: destino any.

any — Cualquiera: cualquier IP o cualquier puerto.

LAS OPCIONES

Entre paréntesis, y cada una acaba en ;

Opción	Qué significa	Ejemplo
msg	el nombre de la alerta (obligatoria)	`msg:"LAB ping desde Internet";`
sid · rev	número único (las nuestras, ≥ 1000000) y versión	`sid:1000001; rev:1;`
itype	tipo de ICMP: 8 = petición de ping	`itype:8;`
content · nocase	busca este texto · da igual mayúsculas	`content:"union"; nocase;`
http.uri	el content siguiente, búscalo en la URL	`http.uri; content:"union";`
flags	S = paquete que abre conexión (SYN)	`flags:S;`
threshold	avisa cada N veces en X segundos	`threshold: type threshold, track by_src, count 20, seconds 10;`

SI FALTA UN ; O UNA , — Suricata no carga la regla. ./lab.sh reglas te dice en qué línea está el error.

CÓMO SE TRABAJA

Cuatro pasos, siempre los mismos

1 — Escribe

`firewall/suricata/local.rules`

2 — Comprueba y carga

`./lab.sh reglas`

3 — Ataca

`./lab.sh internet`

4 — Mira

`./lab.sh alertas`

`./lab.sh reglas` (con un error: falta la coma entre «count 20» y «seconds 10»)

```
▶️ Comprobando firewall/suricata/local.rules…
✘ Hay un error en las reglas. No he cargado nada: sigue Suricata con las de antes.
E: detect-threshold: pcre_exec parse error, ret -1, string type threshold, track by_src, count 20 seconds 10
E: detect: error parsing signature "alert tcp $EXTERNAL_NET any -> ..." from file /lab/suricata/local.rules at line 17
```

SI TODO VA BIEN — ✔️ Reglas cargadas (3 reglas, modo IDS). Míralas trabajar con ./lab.sh alertas

DEMO 1 · EL PING

Provocar la alerta y leerla

```
./lab.sh internet

ping -c 2 10.10.20.10
64 bytes from 10.10.20.10: icmp_seq=1 ttl=63 time=0.119 ms
64 bytes from 10.10.20.10: icmp_seq=2 ttl=63 time=0.162 ms
```

```
./lab.sh alertas

22:13:49  203.0.113.50 → 10.10.20.10:-  LAB ping desde Internet
22:13:50  203.0.113.50 → 10.10.20.10:-  LAB ping desde Internet
```

hora · quién → a quién:puerto · nombre de la regla (msg)

`signature_id` = el sid de tu regla · **`signature`** = su msg

```
/var/log/suricata/eve.json (una alerta)

{
  "timestamp": "2026-10-28T22:14:12…",
  "event_type": "alert",
  "src_ip": "203.0.113.50",
  "dest_ip": "10.10.20.10",
  "proto": "ICMP",
  "alert": {
    "action": "allowed",
    "signature_id": 1000001,
    "signature": "LAB ping desde Internet",
    "severity": 3
  }
}
```

action: allowed — El IDS solo ha mirado: el ping ha llegado igual. Avisar no es parar.

DEMO 2 · EL ESCANEO DE PUERTOS

Una regla nueva: muchos SYN seguidos desde la misma IP

firewall/suricata/local.rules (añadida)

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB escaneo de puertos"; flags:S;
    threshold: type threshold, track by_src, count 20, seconds 10; sid:1000002; rev:1;)
```

(en el fichero, todo en UNA línea)

./lab.sh internet

```
nmap -sS -Pn -n -p 1-100 10.10.20.10

Not shown: 99 closed tcp ports (reset)
PORT   STATE SERVICE
80/tcp open  http
```

./lab.sh alertas

```
22:15:22  203.0.113.50 → 10.10.20.10:29  LAB escaneo…
22:15:22  203.0.113.50 → 10.10.20.10:72  LAB escaneo…
22:15:22  203.0.113.50 → 10.10.20.10:94  LAB escaneo…
22:15:22  203.0.113.50 → 10.10.20.10:56  LAB escaneo…
22:15:22  203.0.113.50 → 10.10.20.10:37  LAB escaneo…
```

¿POR QUÉ 5 ALERTAS? — 100 puertos = 100 SYN. La regla avisa una vez cada 20 → 5 alertas (y no 100).

✍️ TU TURNO

Avisa si en la URL aparece «union»

1	Acción y protocolo	`alert http  (Suricata entiende la web)`
2	Origen → destino	`$EXTERNAL_NET any -> any any  (por el DNAT)`
3	Dónde buscar	`http.uri;  → en la URL`
4	Qué buscar	`content:"union"; nocase;`
5	Nombre y número	`msg:"LAB inyeccion SQL en la URL"; sid:1000003; rev:1;`

para probarla, desde ./lab.sh internet

```
curl "http://203.0.113.254/?id=1%20UNION%20SELECT%20user,pass%20FROM%20clientes"
# después: ./lab.sh alertas  →  … 203.0.113.50 → 203.0.113.254:80  LAB inyeccion SQL en la URL
```

COMPRUEBA — ./lab.sh comprobar ud5 → las pruebas de Suricata en verde

La misma regla, con drop, y Suricata en medio

firewall/suricata/local.rules (solo para la demo)

```
drop icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB ping cortado"; itype:8; sid:1000009; rev:1;)
```

terminal

```
./lab.sh ips      # Suricata en medio
./lab.sh reglas
./lab.sh internet  →  ping -c 3 10.10.20.10
```

lo que sale

```
3 packets transmitted, 0 received, 100% packet loss

./lab.sh alertas
22:18:25  … [BLOQUEADO] LAB ping cortado
```

drop en modo IDS

No para nada: solo mira una copia. Hace falta ./lab.sh ips.

Un drop mal hecho

Deja fuera a los clientes. Primero IDS; luego, drop.

AL ACABAR — Borra la regla de la demo y vuelve a ./lab.sh ids.

CUANDO SE EQUIVOCA

Falso positivo y falso negativo

	¿Salta la alarma? SÍ	¿Salta la alarma? NO
Era un ataque	**Verdadero positivo** Bien: el escaneo → alerta	**Falso negativo** Lo PEOR: no nos enteramos
No era nada	**Falso positivo** Molesta: un cliente bloqueado	**Verdadero negativo** Bien: tráfico normal, sin ruido

AJUSTAR — Más estricto = más falsos positivos · menos estricto = más falsos negativos.

El dependiente que lee cada pedido



ModSecurity

el dependiente

Lee la petición web ENTERA

- la URL: /?id=1' OR '1'='1
- los campos de los formularios
- las cabeceras y las cookies

y la compara con la OWASP CRS: reglas gratuitas contra los ataques web más comunes (OWASP Top 10)

Inyección SQL

```
?id=1' OR '1'='1
```

XSS

```
<script>alert(1)</script>
```

Recorrer carpetas

```
../../etc/passwd
```

OWASP — Fundación que publica la lista de los 10 problemas web más comunes (Top 10) y la CRS.

CÓMO DECIDE

No bloquea por una regla: suma puntos

3	5	≥ 5 → 403
aviso	crítico	umbral

Cada regla que salta suma puntos. Si el total llega al umbral, el WAF contesta 403 (prohibido).

```
./lab.sh bloqueos   (WAF en modo On)

22:15:27  GET /?id=1  → 200
    [920350] Host header is a numeric IP address                ← 3 puntos: pasa

22:15:27  GET /?id=1%27%20OR%20%271%27=%271  → 403  BLOQUEADA
    [920350] Host header is a numeric IP address                ← 3
    [942100] SQL Injection Attack Detected via libinjection     ← 5
    [949110] Inbound Anomaly Score Exceeded (Total Score: 8)    ← 8 ≥ 5: fuera
```

%27 = ' — En la URL las comillas y los espacios van codificados: %27 es ' y %20 es un espacio.

LA TRAMPA

DetectionOnly apunta… y deja pasar

waf/waf.env (como arranca)

```
MODSEC_RULE_ENGINE=DetectionOnly
```

waf/waf.env (✍ 3)

```
MODSEC_RULE_ENGINE=On
```

./lab.sh bloqueos (DetectionOnly)

```
GET /?id=1%27%20OR%20%271%27=%271  → 200
  [942100] SQL Injection Attack…
  [949110] … (Total Score: 8)
```

./lab.sh bloqueos (On)

```
GET /?id=1%27%20OR%20%271%27=%271  → 403  BLOQUEADA
  [942100] SQL Injection Attack…
  [949110] … (Total Score: 8)
```

200: la tienda lo recibe

El ataque está apuntado… y ha pasado.

403: no llega a la tienda

Después de cambiarlo: ./lab.sh waf

MODOS — Off = apagado · DetectionOnly = solo apunta · On = bloquea.

UNA EXCEPCIÓN BIEN HECHA

Lo más pequeña posible

waf/plugins/excepciones-before.conf

```
SecRule REQUEST_URI "@beginsWith /contacto" "id:1000,phase:1,pass,nolog,ctl:ruleRemoveTargetByTag=attack-sqli;ARGS:comentario"
```

`SecRule REQUEST_URI "@beginsWith /contacto"` → SOLO si la página empieza por /contacto

`id:1000,phase:1,pass,nolog` → número de la regla · antes de mirar nada · deja seguir · sin apuntar

`ctl:ruleRemoveTargetByTag=attack-sqli` → quita las reglas de inyección SQL…

`;ARGS:comentario` → …SOLO para el campo «comentario»

Mal

MODSEC_RULE_ENGINE=Off · quitar la regla en toda la web

Bien

comentario en /contacto: 200 · nombre u otra página: 403

DESPUÉS — ./lab.sh waf y ./lab.sh comprobar ud5

CAPAS, NO RIVALES

El mismo ataque, visto por los dos

```
./lab.sh internet
curl "http://203.0.113.254/?id=1%20UNION%20SELECT%20user,pass%20FROM%20clientes"     →  403
```

Cortafuegos (UD3)

Quién entra y por qué puerta

`puerto 80: pasa`

Suricata (IDS)

Todo el tráfico, cualquier protocolo

`alerta: LAB inyeccion SQL`

WAF

La web, al detalle

`403: no llega a la tienda`

CON HTTPS — Suricata en la puerta no ve dentro (va cifrado). El WAF sí: es él quien descifra.

ERRORES TÍPICOS

Los cuatro que más vais a ver

Regla sin sid

Suricata no la carga

```
alert icmp any any -> any any (msg:"ping";)
```

```
(msg:"ping"; sid:1000001; rev:1;)
```

drop con Suricata en modo IDS

Avisa, pero no corta nada

```
drop …  +  ./lab.sh ids
```

```
drop …  +  ./lab.sh ips
```

El WAF en DetectionOnly

Apunta el ataque y lo deja pasar (200)

```
MODSEC_RULE_ENGINE=DetectionOnly
```

```
MODSEC_RULE_ENGINE=On
```

Excepción para toda la web

Arregla un formulario y abre todos

```
ctl:ruleRemoveByTag=attack-sqli
```

```
solo /contacto y ;ARGS:comentario
```

CONTROL DE COMPRENSIÓN

Preguntas de control y debate

1. Un IDS detecta un ataque. ¿Lo para? ¿Por qué?
2. alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"SSH"; sid:1000010; rev:1;) — ¿qué dice cada parte?
3. El WAF está en DetectionOnly y llega ?id=1' OR '1'='1. ¿Qué código recibe el atacante?
4. El WAF bloquea el comentario de una clienta. ¿Qué es? ¿Cómo lo arreglas sin abrir la puerta?
5. ¿Por qué el cortafuegos de la UD3 no para una inyección SQL?

LO QUE NOS LLEVAMOS HOY

Resumen de puntos clave

- IDS = cámaras: avisa · IPS = corta el paso (drop)
- Regla: acción · cabecera · (opciones; sid; rev;)
- local.rules → ./lab.sh reglas → atacar → ./lab.sh alertas
- Falso positivo molesta · falso negativo, lo peor
- WAF + CRS: suma puntos; ≥ 5 → 403
- DetectionOnly NO bloquea · excepciones, lo más pequeñas posible

Las cámaras avisan, el vigilante corta y el dependiente lee cada pedido.

LO SIGUIENTE

UD6 · Cerrar puertas y ventanas

- El servidor del mismo laboratorio
- SSH sin root y sin contraseña
- Fail2ban: el portero
- Lynis: el inspector